

Cognizant-Up Grad Training Program

UGE_DOTNET FSD (Python)

Weekly Submission Document Trainee Information

Name: YARRAM VINAY KUMAR

Email: vinaykumaryarram692@gmail.com

Batch Name: B1 Python

Course: UGE DOTNet FSD (Python)

Training Duration: 16 Feb 2026-26" May 2026 Session

Timings: 9:30 AM-6:30 PM

Faculty: Kunal

Week-7

Q1: Notification System (Strategy + DI)

✓ Question (Short)

Design a system to send Email/SMS/Push notifications

- No `if-else`
- Add new types easily
- Retry once if fail

// Step 1: Common Interface

```
public interface INotification
```

```
{
```

```
    Task SendAsync(string message);
```

```
}
```

// Step 2: Implementations

```
public class EmailNotification : INotification
```

```
{
```

```
    public async Task SendAsync(string message)
```

```
    {
```

```
        Console.WriteLine("Email Sent: " + message);
```

```

        await Task.CompletedTask;
    }
}

```

```

public class SmsNotification : INotification
{
    public async Task SendAsync(string message)
    {
        Console.WriteLine("SMS Sent: " + message);
        await Task.CompletedTask;
    }
}

```

```

// Step 3: Factory (No if-else → use Dictionary)
public class NotificationFactory
{
    private readonly Dictionary<string, INotification> _services;

    public NotificationFactory(IEnumerable<INotification> services)
    {
        _services = services.ToDictionary(s => s.GetType().Name);
    }

    public INotification Get(string type)
    {
        if (_services.ContainsKey(type))
            return _services[type];

        throw new Exception("Invalid Type");
    }
}

```

```

// Step 4: Service with Retry
public class NotificationService
{
    private readonly NotificationFactory _factory;

    public NotificationService(NotificationFactory factory)
    {
        _factory = factory;
    }

    public async Task Send(string type, string msg)
    {
        var service = _factory.Get(type);

        try
        {

```

```

        await service.SendAsync(msg);
    }
    catch
    {
        Console.WriteLine("Retrying...");
        await service.SendAsync(msg);
    }
}
}

```

Q2: Thread-Safe Cache

✓ Question (Short)

Design cache:

- Set(key, value, expiry)
- Thread-safe
- Auto remove expired

```

public class CacheItem
{
    public string Value { get; set; }
    public DateTime Expiry { get; set; }
}

```

```

public class CacheService
{
    private static readonly object _lock = new();
    private static Dictionary<string, CacheItem> _cache = new();

    public void Set(string key, string value, int seconds)
    {
        lock (_lock)
        {
            _cache[key] = new CacheItem
            {
                Value = value,
                Expiry = DateTime.Now.AddSeconds(seconds)
            };
        }
    }
}

```

```

public string Get(string key)
{
    lock (_lock)
    {
        if (_cache.ContainsKey(key))
        {
            var item = _cache[key];

            if (item.Expiry > DateTime.Now)
                return item.Value;

            _cache.Remove(key);
        }
        return null;
    }
}

```

Q3: JWT Authentication (Basic)

✓ Question (Short)

Login → generate JWT
Protected API → validate token

```

// Generate Token
public string GenerateToken(string userId)
{
    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes("secretkey123"));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        claims: new[] { new Claim("UserId", userId) },
        expires: DateTime.Now.AddMinutes(15),
        signingCredentials: creds);

    return new JwtSecurityTokenHandler().WriteToken(token);
}

```

Q4: Rate Limiter

✓ Question (Short)

Limit 5 requests per 10 sec

```

public class RateLimiter
{
    private Dictionary<string, List<DateTime>> _requests = new();

    public bool IsAllowed(string user)
    {
        if (!_requests.ContainsKey(user))
            _requests[user] = new List<DateTime>();

        var now = DateTime.Now;

        _requests[user].RemoveAll(t => (now - t).TotalSeconds > 10);

        if (_requests[user].Count >= 5)
            return false;

        _requests[user].Add(now);
        return true;
    }
}

```

Q5: Deadlock Example

✓ Code (Deadlock)

```

object lock1 = new object();
object lock2 = new object();

```

```

void Method1()
{
    lock (lock1)
    {
        Thread.Sleep(100);
        lock (lock2)
        {
            Console.WriteLine("Method1");
        }
    }
}

```

```

void Method2()
{
    lock (lock2)
    {
        Thread.Sleep(100);
        lock (lock1)

```

```

    {
        Console.WriteLine("Method2");
    }
}

```

Q6: Custom HashMap (Simple)

```

public class MyHashMap
{
    private List<(int key, string value)>[] buckets;

    public MyHashMap(int size)
    {
        buckets = new List<(int, string)>[size];
    }

    private int Hash(int key) => key % buckets.Length;

    public void Add(int key, string value)
    {
        int index = Hash(key);

        if (buckets[index] == null)
            buckets[index] = new List<(int, string)>();

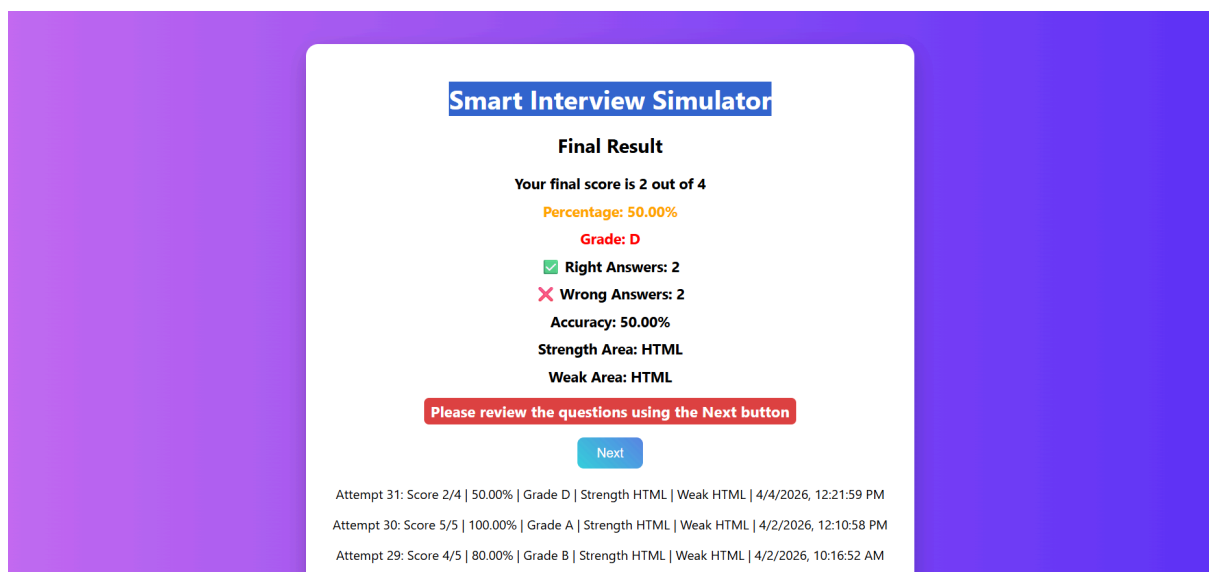
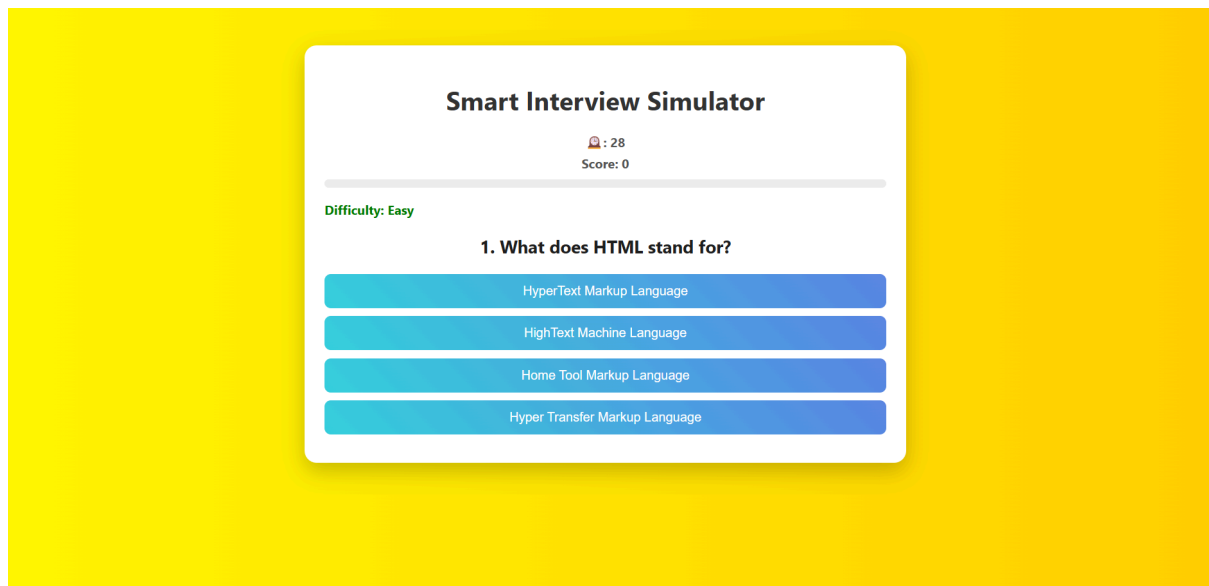
        buckets[index].Add((key, value));
    }

    public string Get(int key)
    {
        int index = Hash(key);

        if (buckets[index] != null)
        {
            foreach (var item in buckets[index])
            {
                if (item.key == key)
                    return item.value;
            }
        }
        return null;
    }
}

```

Smart Interview Simulator



Full-Stack Healthcare Appointment System (ASP.NET Core + SQL)

Problem Statement

An emerging healthcare startup wants to transform its basic appointment booking website into a full-stack enterprise-grade application using ASP.NET Core 8, C#, and SQL Server.

Develop a multi-layered web application that supports doctor management, appointment scheduling, patient records, and secure data storage using MVC architecture and REST APIs.

The system must:

- Persist data in a relational database
 - Use Entity Framework Core for ORM
 - Follow SOLID principles and design patterns
 - Implement authentication-ready structure and secure coding practices
 - Support scalable and maintainable architecture
-



Core Features to Implement

Application Modules

- Dashboard Module – Displays upcoming appointments and statistics
 - Doctors Module – CRUD operations for doctors and specialization
 - Appointments Module – Book, update, cancel appointments
 - Patient Profile Module – Stores patient details and history
 - API Module – REST APIs for doctors, appointments, and patients
-



Functional Requirements



Database & SQL (Detailed Schema Required)

 **You MUST create the following tables:**

1. Patients

```
PatientId (PK, INT, Identity)
FullName (VARCHAR)
Email (UNIQUE)
PhoneNumber
CreatedAt (DATETIME)
```

2. Doctors

DoctorId (PK)
FullName
Specialization
Email

3. Appointments

AppointmentId (PK)
PatientId (FK → Patients.PatientId)
DoctorId (FK → Doctors.DoctorId)
AppointmentDate (DATETIME)
Status (Scheduled / Completed / Cancelled)

4. MedicalRecords

RecordId (PK)
PatientId (FK)
DoctorId (FK)
Diagnosis
Prescription
CreatedAt

5. Departments

DepartmentId (PK)
Name

6. DoctorDepartments

Id (PK)
DoctorId (FK)
DepartmentId (FK)

Relationships (MANDATORY)

- One Patient → Many Appointments
 - One Doctor → Many Appointments
 - One Patient → Many Medical Records
 - Many Doctors ↔ Many Departments
-

SQL Requirements

You MUST write queries for:

- SELECT, WHERE, ORDER BY
- INNER JOIN, LEFT JOIN
- GROUP BY (appointments per doctor)
- Subqueries (patients with highest visits)
- UNION (combine doctor & patient emails)
- INSERT, UPDATE, DELETE

Swagger UI

Patients - HealthcareApp

ChatGPT

localhost:5055/PatientsMvc

Summarize

Healthcare App

Patients

Doctors

Appointments

Medical Records

Patients

[Add New Patient](#)

Name	Email	Phone	Actions
John Doe	johndoe@example.com	5854125421	Details Edit
John Doe	johndoe@example.com	555-1234	Details Edit
Jane Smith	janesmith@example.com	555-5678	Details Edit
Michael Johnson	michael.johnson@example.com	555-9012	Details Edit
Emily Davis	emily.davis@example.com	555-3456	Details Edit
abfghjd	abcdasd@example.com	4561239635	Details Edit
vijaya	user@example.com	8542587451	Details Edit
Meeniga Suseela	meenigasuseela@gmail.com	85550 20418	Details Edit
asdq	asdf@example.com	1542689542	Details Edit
string	user@example.com	string	Details Edit
vijji	vijji@example.com	87542165321	Details Edit

© 2026 HealthcareApp. All rights reserved.

Swagger UI

Patients - HealthcareApp

ChatGPT

localhost:5055/PatientsMvc/Details/1

Summarize

Healthcare App

Patients

Doctors

Appointments

Medical Records

Patient Details

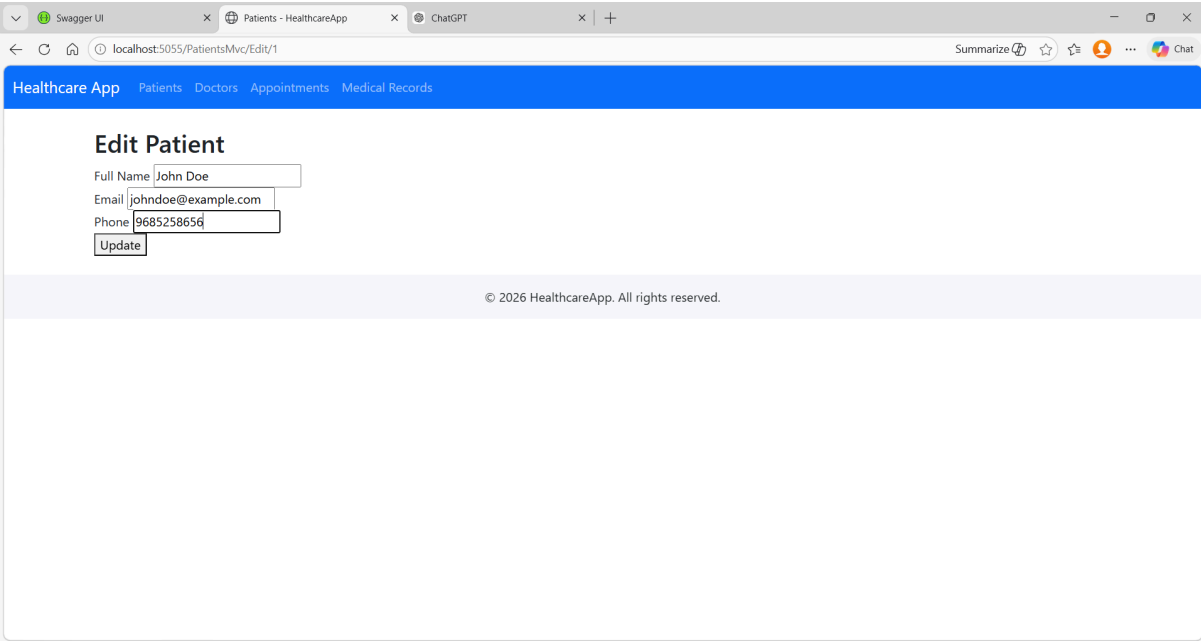
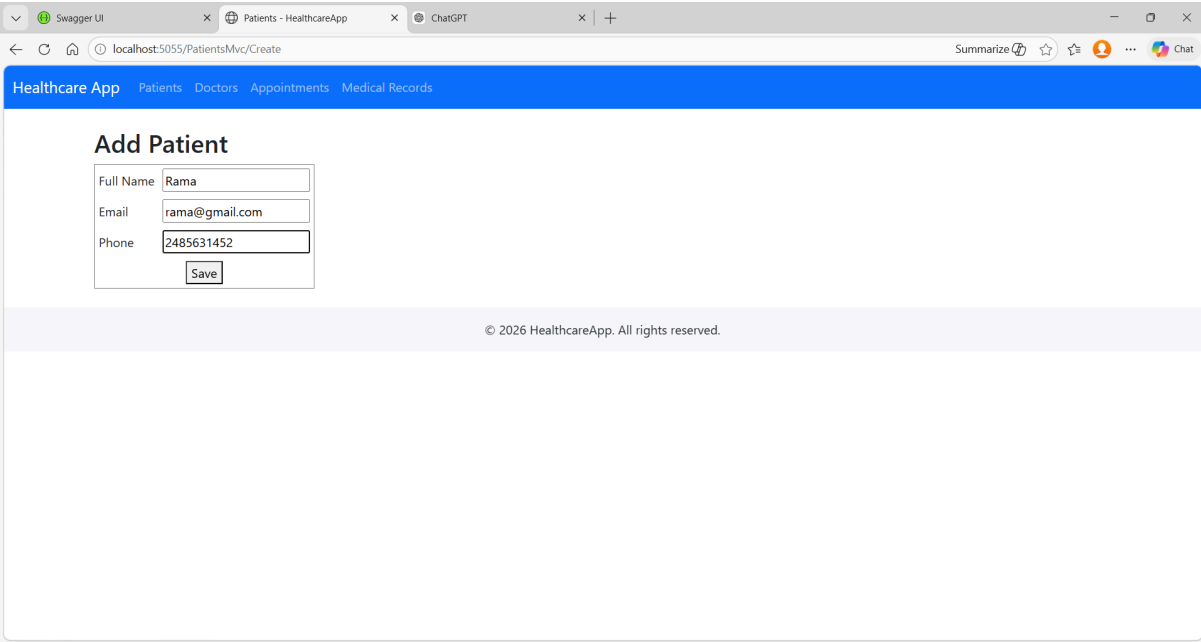
Name: John Doe

Email: johndoe@example.com

Phone: 9685258656

[Back](#)

© 2026 HealthcareApp. All rights reserved.



Swagger UIDoctors - HealthcareAppChatGPT

localhost:5055/DoctorsMvcSummarize

Healthcare AppPatientsDoctorsAppointmentsMedical Records

Doctors

[Add New Doctor](#)

Name	Specialization	Email	Actions
Sravani	ENT	sravani6788@gmail.com	Details Edit Delete
Suresh	newurologist	suresh236788@gmail.com	Details Edit Delete
sarala	Cardiologist	sarala345@gmail.com	Details Edit Delete
rani	cardiologist	rani@gmail.com	Details Edit Delete
kiranw	ENT	kiran2345@gmail.com	Details Edit Delete
Suseela	ENT	suseela@gmail.com	Details Edit Delete
rani	cardiologist	rani@gmail.com	Details Edit Delete

© 2026 HealthcareApp. All rights reserved.

Swagger UIDoctors - HealthcareAppChatGPT

localhost:5055/DoctorsMvc/CreateSummarize

Healthcare AppPatientsDoctorsAppointmentsMedical Records

Add Doctor

Full Name

Ramana

Specialization

ENT

Email

ramana234@gmail.com

Save

© 2026 HealthcareApp. All rights reserved.

